Case Study 10: Tuple-Based Product Catalogue Audit

1. Problem Statement

Develop a Python program to audit a product catalogue using tuples. The program should identify duplicate product IDs, find low-stock products, identify expensive products, and calculate the total inventory value.

2. Objectives

- Store product details using tuples.
- Use tuple unpacking to access product information.
- Identify duplicate product IDs.
- Find products with low stock.
- Identify expensive products.
- Calculate the total value of available inventory.

3. Assumptions

1. Each product contains a product ID, product name, category, price, and stock quantity.
2. Product IDs should be unique.
3. A product is considered low stock when its stock quantity is 5 or less.
4. A product is considered expensive when its price is ₹5,000 or more.
5. Inventory value is calculated as `price × stock quantity`.
6. The sample product records can be changed to test different cases.

4. Algorithm

1. Start the program.
2. Store product records as tuples inside a list.
3. Initialize the total inventory value to zero.
4. Display each product's details using tuple unpacking.
5. Calculate the inventory value of each product.
6. Check whether the product ID is duplicated.
7. Identify low-stock and expensive products.

8. Display the audit report and total inventory value.
9. Stop the program.

5. Flowchart

START

Store product records as tuples in a list

Read product details using tuple unpacking

Calculate inventory value and check stock and price

Find duplicate product IDs

Display the audit report and total inventory value

STOP

6. Pseudocode

```
None
BEGIN

    Store product records in a list of tuples
    Set total_value = 0
    Set duplicate_ids = empty list

    FOR each product in products
        Unpack product details
        Calculate price × stock
        Add the result to total_value

        IF stock <= 5
            Display low-stock product
        END IF

        IF price >= 5000
            Display expensive product
        END IF
```

```
    END FOR

    Compare product IDs to find duplicates
    Display duplicate IDs
    Display total inventory value

END
```

7. Python Source Code

Save this file as `product_catalogue_audit.py`.

```
None

# Case Study 10: Tuple-Based Product Catalogue Audit

# Each tuple contains:
# (Product ID, Name, Category, Price, Stock)

products = [
    ("P101", "Keyboard", "Accessories", 750, 12),
    ("P102", "Mouse", "Accessories", 450, 5),
    ("P103", "Monitor", "Electronics", 8500, 3),
    ("P102", "USB Cable", "Accessories", 250, 20),
    ("P104", "Laptop Stand", "Accessories", 1200, 0)
]

total_value = 0
duplicate_ids = []

print("PRODUCT CATALOGUE AUDIT")
print("-" * 60)

# Display product details and perform checks
for product in products:
```

```
    product_id, name, category, price, stock = product

    inventory_value = price * stock
    total_value += inventory_value

    print(
        product_id, "|", name, "|", category,
        "| Price: Rs.", price,
        "| Stock:", stock,
        "| Value: Rs.", inventory_value
    )

print("\nLOW-STOCK PRODUCTS (Stock <= 5):")

for product in products:
    product_id, name, category, price, stock = product

    if stock <= 5:
        print(product_id, "-", name, "- Stock:", stock)

print("\nEXPENSIVE PRODUCTS (Price >= Rs. 5000):")

for product in products:
    product_id, name, category, price, stock = product

    if price >= 5000:
        print(product_id, "-", name, "- Price: Rs.", price)

# Find duplicate product IDs
for i in range(len(products)):
    for j in range(i + 1, len(products)):
        if products[i][0] == products[j][0]:
            if products[i][0] not in duplicate_ids:
                duplicate_ids.append(products[i][0])

print("\nDUPLICATE PRODUCT IDs:")
```

```
if len(duplicate_ids) > 0:
    for product_id in duplicate_ids:
        print(product_id)
else:
    print("No duplicate IDs found.")

print("\nTOTAL INVENTORY VALUE: Rs.", total_value)
print("AUDIT COMPLETED")
```

8. Test Cases

Test case	Input condition	Expected result
1	Stock is 5	Product appears under low stock
2	Price is ₹8,500	Product appears under expensive products
3	Two products have ID `P102`	Duplicate ID is detected
4	Stock is 0	Product appears under low stock
5	All sample products	Total inventory value is ₹41,750

9. Output Screenshot

```python
# Case Study 10: Tuple-Based Product Catalogue Audit

# Each tuple contains:
# (Product ID, Name, Category, Price, Stock)

products = [
    ("P101", "Keyboard", "Accessories", 750, 12),
    ("P102", "Mouse", "Accessories", 450, 5),
    ("P103", "Monitor", "Electronics", 8500, 3),
    ("P102", "USB Cable", "Accessories", 250, 20),
    ("P104", "Laptop Stand", "Accessories", 1200, 0)
]

total_value = 0
duplicate_ids = []

print("PRODUCT CATALOGUE AUDIT")
print("-" * 60)

# Display product details and perform checks
for product in products:
    product_id, name, category, price, stock = product

    inventory_value = price * stock
    total_value += inventory_value

    print(
        product_id, "|", name, "|", category,
        "| Price: Rs.", price,
        "| Stock:", stock,
        "| Value: Rs.", inventory_value
    )

print("\nLOW-STOCK PRODUCTS (Stock <= 5):")

for product in products:
    product_id, name, category, price, stock = product

    if stock <= 5:
        print(product_id, "-", name, "- Stock:", stock)

print("\nEXPENSIVE PRODUCTS (Price >= Rs. 5000):")

for product in products:
    product_id, name, category, price, stock = product

    if price >= 5000:
        print(product_id, "-", name, "- Price: Rs.", price)
```

```
# Find duplicate product IDs
for i in range(len(products)):
    for j in range(i + 1, len(products)):
        if products[i][0] == products[j][0]:
            if products[i][0] not in duplicate_ids:
                duplicate_ids.append(products[i][0])

print("\nDUPLICATE PRODUCT IDs:")

if len(duplicate_ids) > 0:
    for product_id in duplicate_ids:
        print(product_id)
else:
    print("No duplicate IDs found.")

print("\nTOTAL INVENTORY VALUE: Rs.", total_value)
print("AUDIT COMPLETED")
```

```
>
========= RESTART: C:/Users/YOGITHA/OneDrive/Desktop/10.TUPLE based.py =========
PRODUCT CATALOGUE AUDIT
------------------------------------------------------------
P101 | Keyboard | Accessories | Price: Rs. 750 | Stock: 12 | Value: Rs. 9000
P102 | Mouse | Accessories | Price: Rs. 450 | Stock: 5 | Value: Rs. 2250
P103 | Monitor | Electronics | Price: Rs. 8500 | Stock: 3 | Value: Rs. 25500
P102 | USB Cable | Accessories | Price: Rs. 250 | Stock: 20 | Value: Rs. 5000
P104 | Laptop Stand | Accessories | Price: Rs. 1200 | Stock: 0 | Value: Rs. 0

LOW-STOCK PRODUCTS (Stock <= 5):
P102 - Mouse - Stock: 5
P103 - Monitor - Stock: 3
P104 - Laptop Stand - Stock: 0

EXPENSIVE PRODUCTS (Price >= Rs. 5000):
P103 - Monitor - Price: Rs. 8500

DUPLICATE PRODUCT IDs:
P102

TOTAL INVENTORY VALUE: Rs. 41750
AUDIT COMPLETED
>
```

10. Conclusion

The Tuple-Based Product Catalogue Audit program demonstrates how tuples can store product details and how tuple unpacking can access individual values. It identifies duplicate product IDs,

low-stock products, and expensive products, and calculates the total inventory value. This program helps maintain an organized and accurate product catalogue.